

Grid-based wavepacket dynamics:

NaF photodissociation on two coupled diabatic states

Computational Spectroscopy School — programming exercise

Duration: this is the main programming exercise for the 2.5-hour session. **Prerequisites:** basic quantum mechanics and Python (`numpy`, `scipy`, `matplotlib`). **Goal:** implement your own grid-based, split-operator wavepacket propagation for a two-state curve-crossing problem.

1. The physical system

Two diabatic electronic states of NaF, $|0\rangle$ (ionic character) and $|1\rangle$ (covalent character), are coupled both by a diabatic coupling near their crossing and by a laser pulse. Nuclear motion is treated on a single coordinate x (the Na–F distance) with reduced mass $\mu = m_{\text{Na}}m_{\text{F}}/(m_{\text{Na}} + m_{\text{F}})$. All quantities are in atomic units.

The nuclear wavefunction is a 2-component spinor $\Psi(x, t) = (\psi_1(x, t), \psi_2(x, t))^T$, normalized as $\int (|\psi_1|^2 + |\psi_2|^2) dx = 1$, evolving under the explicitly time-dependent Hamiltonian

$$H(x, t) = -\frac{1}{2\mu} \frac{\partial^2}{\partial x^2} + \begin{pmatrix} V_1(x) & V_c(x) - \mu_{12}(x) E(t) \\ V_c(x) - \mu_{12}(x) E(t) & V_1(x) \end{pmatrix}. \quad (1)$$

Here V_1, V_2 are the diabatic potential curves, $V_c(x)$ is a fixed, spatially localized non-adiabatic coupling (independent of the laser), $\mu_{12}(x)$ is the transition dipole moment, and $E(t)$ is the laser field — the source of the explicit time dependence, on top of H 's x -dependence.

2. Learning objectives

By the end of this exercise you should be able to: represent a wavefunction and its momentum-space partner on a discrete grid, and explain the Nyquist–Shannon relationship between them; derive and implement the symmetric split-operator propagator for a Hamiltonian that is time-dependent through an external field; handle a multi-state (here: 2-state) potential step as a 2×2 matrix exponential problem, both the transparent way and a fast vectorized way; and design and run the numerical validation checks (normalization, time-step convergence, grid convergence, boundary effects) that tell you whether a wavepacket calculation can be trusted.

3. Provided data (naf_data/)

File	Contents
<code>x.csv</code>	the position grid x_i (bohr), uniformly spaced
<code>d0.csv</code>	$V_1(x_i)$, diabat $ 0\rangle$ (hartree)
<code>d1.csv</code>	$V_2(x_i)$, diabat $ 1\rangle$ (hartree)
<code>dc.csv</code>	$V_c(x_i)$, fixed diabatic coupling (hartree)
<code>dipole_01.csv</code>	$\mu_{12}(x_i)$, transition dipole moment (a.u.)
<code>wf0.csv</code>	the initial wavefunction on state $ 0\rangle$ at $t = 0$

All six files share the same length and grid. `wf0.csv` is a *given* initial condition — check its normalization, but do not assume it is the harmonic ground state of V_1 (it need not be, and for this particular V_1 curve, isn't).

4. Implementation steps

Step 0 — Define constants and conversion factors

- $\hbar = 1.0$ # Planck's constant
- $m_e = 1822.89$ # one atomic mass unit in electron masses
- $a_0 = 0.5292$ # Bohr radius in Angstrom (1 Angstrom = 1.89 a.u.)
- $\text{au} \rightarrow \text{ev} = 27.21$ # 1 a.u. = 27.21 eV
- $\text{au} \rightarrow \text{fs} = 1 / 41.34$ # 1 a.u. = 0.0242 fs

Step 1 — Load the data and look at it before writing any physics

Load all six files. Plot V_1, V_2, V_c together and identify (numerically, not by eye) the approximate diabatic crossing point x_c where $V_1(x_c) \approx V_2(x_c)$, and where $V_c(x)$ peaks. Plot $\mu_{12}(x)$ and $|\psi(x, 0)|^2$. Confirm $\int |\psi(x, 0)|^2 dx = 1$ to floating-point precision (renormalize once if it's off by more than $\sim 10^{-4}$ – that's a data-precision artifact, not physics).

Step 2 — Build the momentum grid

With N_x points spaced by Δx , the largest resolvable wavenumber is the Nyquist limit $k_{\max} = \pi/\Delta x$. Build the momentum grid consistently with whatever FFT convention your language/library uses (e.g. `scipy.fft.fftfreq`, scaled by 2π) — position space and momentum space carry exactly the same amount of information, N_x numbers either way, so under-resolving one under-resolves the other.

Step 3 — Build the laser field

In the cleanest case, laser fields with a finite temporal width are expressed via the convolution of a Gaussian envelope with a cosine function:

$$\varepsilon(t) = E_0 \exp \left[-\frac{(t - t_0)^2}{2\sigma_t^2} \right] \cos [\omega(t - t_0)]. \quad (2)$$

First, define a time axis with the following parameters:

- $dt = 5$ # Time step
- $nt = 4000$ # Number of time steps
- $t = \text{np.arange}(0, dt*nt, dt)$ # Time axis

Then, generate a laser field according to Eq. 2 with the following parameters:

- $E_0 = 0.025$ # Maximum field intensity
- $\sigma_t = 5*41.34$ # 5 fs FWHM
- $\omega = V_2(x) - V_1(x)$ (energy gap where the initial wavepacket sits) # Field frequency
- $t_0 = 900$ # time of maximum pulse intensity

Plot the laser field in time and frequency domain.

Step 4 — Build the potential matrix

The potential part is a 2×2 matrix at every coordinate-grid point. Load V_1 and V_2 into the diagonal elements of the $N_x \times 2 \times 2$ tensor. The off-diagonal coupling element is time dependent and must be updated at each time step of the propagation.

Step 5 — Build the kinetic energy operator

The nuclear kinetic-energy operator is $T(k) = \frac{k^2}{2M}$ with the momentum axis k and the reduced mass of NaF is $M = \frac{m_{\text{Na}}m_{\text{F}}}{m_{\text{Na}}+m_{\text{F}}}$. Remember to use atomic units.

Step 6 — The propagator: symmetric split-operator

Familiarize yourself (or even derive) the second-order split-operator step

$$\Psi(x, t + \Delta t) \approx e^{-iT\Delta t/2} e^{-iV(x, t + \Delta t/2)\Delta t} e^{-iT\Delta t/2} \Psi(x, t), \quad (3)$$

evaluating the potential at the *midpoint* time. *Optional:* Understand why the local error is $\mathcal{O}(\Delta t^3)$ (hint: the leading commutator term $[T, [T, V]]$) the extra source of error specific to this problem: V depends on t , not just x , through $E(t)$.

Step 7 — Set up the propagation loop

Loop over each point in your time axis. To propagate $\Psi(x, t) \rightarrow \Psi(x, t + \Delta t)$, do the following steps:

Step 8 — Kinetic half-step

$T = -\frac{1}{2\mu}\partial_x^2$ is diagonal in momentum space and identical for both electronic states (same nuclear mass). Implement $e^{-iT\Delta t/2}$ as:

- Fourier transform $\Psi(x, t) \rightarrow \Psi(k, t)$
- Multiply by $e^{-ik^2 0.5\Delta t}$
- Inverse transform $\Psi(k, t + 0.5\Delta t) \rightarrow \Psi(x, t + 0.5\Delta t)$.

Apply it to both components of Ψ .

Step 9 — Potential full step

At each grid point x_i , $e^{-iV(x_i, t)\Delta t}$ is the exponential of a 2×2 Hermitian matrix. **First** implement this the transparent way

- Loop over grid points and build the 2×2 matrix explicitly
- Call your library's general matrix-exponential routine (e.g. `scipy.linalg.expm`) on each one

Once you have succeeded replace it with something faster: a 2×2 Hermitian matrix always has a closed-form exponential which you can evaluate with plain array arithmetic over the whole grid at once — no loop, no repeated `expm` calls. Consider the provided jupyter notebook for an implemented solution. Use the fast solution for further production.

Step 10 — Kinetic half-step (repeat step 8)

$T = -\frac{1}{2\mu}\partial_x^2$ is diagonal in momentum space and identical for both electronic states (same nuclear mass). Implement $e^{-iT\Delta t/2}$ as:

- Fourier transform $\Psi(x, t) \rightarrow \Psi(k, t)$
- Multiply by $e^{-ik^2 0.5\Delta t}$
- Inverse transform $\Psi(k, t + 0.5\Delta t) \rightarrow \Psi(x, t + 0.5\Delta t)$.

Apply it to both components of Ψ .

Watch out

Total norm $\int (|\psi_1|^2 + |\psi_2|^2) dx$ should be conserved to machine precision by every step above (kinetic and potential half-steps are each unitary). If it drifts by more than $\sim 10^{-8}$ over your run, something upstream is wrong — a non-Hermitian potential matrix and an inconsistent momentum grid (Step 2) are the two usual suspects. Check this *before* trusting any physics you

compute downstream, not after.

Step 11 — Visualize the wavepacket

Plotting the wavepacket at each time step can severely slow down the propagation. Either plot at a larger increment, or generate a video with the saved wave functions after propagation. Consider the provided jupyter notebook for an implementation to generate a video.

Step 12 — Storage and validation

Record (at least) the total norm, the population on each diabat, and the electronic coherence $\rho_{12}(t) = \int \psi_1^*(x, t) \psi_2(x, t) dx$ at every step. Plot them after propagation.

Numerical checks: **(a)** normalization/population-sum conservation over the whole run; **(b)** *time-step convergence* — halve Δt once or twice and confirm your observables stop changing; **(c)** *grid convergence* — the same test, varying N_x instead; and **(d)** *boundary leakage* — how much probability density sits in the outer few percent of your grid at the end of the run (periodic boundary conditions from the FFT mean a wavepacket that reaches the edge silently reappears on the other side). Report numbers for all four, not just a plot that "looks converged."

5. Bonus (optional, *if time allows*): TRUECARS signal

Optional extension

TRUECARS is, in essence, a direct probe of the electronic coherence $\rho_{12}(t)$ you already computed during the propagation. The signal is given by

$$S(\omega, t) = 2\mathcal{I} \int_{-\infty}^{\infty} dt e^{i\omega(t-T)} \varepsilon_0^*(\omega) \varepsilon_1(t-T) \times \langle \psi(t) | \hat{\alpha} | \psi(t) \rangle$$

$\varepsilon_0^*(\omega)$ is defined in frequency, $\varepsilon_1(t-T)$ in time. $\varepsilon_1(t-T)$ gates the transition polarizability expectation value $\langle \psi(t) | \hat{\alpha} | \psi(t) \rangle$.

For simplicity, we assume the transition polarizability

$$\hat{\alpha} = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$$

to be constant over the nuclear space. Electronic populations do not contribute since they carry no dynamical phase and vanish when taking the imaginary part. The time dependent material property then amounts to:

$$\langle \psi(t) | \hat{\alpha} | \psi(t) \rangle = \left\langle \begin{pmatrix} \Psi_1(t) \\ \Psi_2(t) \end{pmatrix} \middle| \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \middle| \begin{pmatrix} \Psi_1(t) \\ \Psi_2(t) \end{pmatrix} \right\rangle = 2\mathcal{R}\langle \Psi_1(t) | \Psi_2(t) \rangle$$

TRUECARS is implemented in the following way:

- Assign $\alpha = 2\mathcal{R}(\rho_{12})$
- Loop over time. At each step:
- Build $\varepsilon_1(t-T)$ centered at the current time delay and convolute it with α
- Inverse Fourier transform and multiply with $\varepsilon_0^*(\omega)$
- Multiply with $e^{i\omega(t-T)}$
- Take the imaginary part and store

Hint: The final signal $S(\omega, t)$ is a $Nx \times Nk$ matrix.

6. Practical tips

Common pitfalls

Units. Work entirely in atomic units internally; convert to femtoseconds only for axis labels and printed diagnostics, through a single named constant (e.g. `AU_PER_FS = 41.341374575751`) — never a bare number retyped at each use site. **Variable shadowing.** If you call your momentum-grid variable `k`, make sure nothing later in your notebook (a loop index, a bonus-section variable) reuses that name — this is a classic, hard-to-spot source of bugs in exactly this kind of exercise. **Diabatic vs. adiabatic.** Everything in this sheet is diabatic (Eq. 1 has smooth, non-crossing-derivative potential curves and an explicit coupling term); if you ever find yourself wanting to diagonalize H at each x to get "the" potential curves, you have switched representations — that's a legitimate thing to plot for intuition, but do not propagate in it without adjusting the kinetic term accordingly. **Grid vs. momentum resolution.** These are not two independent knobs to tune separately — see Step 2 and the Nyquist relation.

References

1. M. D. Feit, J. A. Fleck Jr., A. Steiger, “Solution of the Schrödinger equation by a spectral method,” *Journal of Computational Physics* **47**, 412–433 (1982).
2. M. D. Feit, J. A. Fleck Jr., “Solution of the Schrödinger equation by a spectral method II: Vibrational energy levels of triatomic molecules,” *Journal of Chemical Physics* **78**, 301 (1983).